

Retro Oasis (Local-First Edition)

Version: 1.2

Date: June 26, 2025

1. Project Overview

1.1. Vision

To create “Retro Oasis,” a modern, bright, and intuitive browser-based gaming platform dedicated to classic and retro games. The entire stack is local-first—no cloud dependencies—providing seamless play, persistent stats, and community interaction via locally stored data.

1.2. Goal

Deliver a fully functional gaming website featuring 10 curated retro games, with: - User profiles (local display names + avatars) - Persistent scoring & stats in a local JSON database - Individual, per-game leaderboards - Responsive, polished UI built on open-source, free tools

1.3. Target Audience

- Casual gamers seeking quick play
 - Millennials & Gen X craving nostalgia
 - Students & office workers looking for short breaks
-

2. Game Library

The platform launches with 10 classic games. Each entry includes UI mockup guidance, core logic summary, feature list, controls mapping, scoring rules, color themes, and accessibility notes.

2.1 Tic-Tac-Toe

- **UI Layout:** 3×3 grid (100×100px cells), centered on screen. Hover/focus highlight: border 2px solid #3B82F6.
- **Core Logic:** 2D array state; check win on rows/columns/diagonals after each move.
- **Features:** Single-player vs CPU (levels: Random, Minimax), two-player local hot-seat.
- **Controls:** Mouse/tap; keyboard arrow navigation + Enter.
- **Scoring:** Win=1, Draw=0.5, Loss=0 stored in stats.
- **Theme:** X = #3B82F6 (blue), O = #F97316 (orange); background #FFFFFF.

- **Accessibility:** ARIA labels on cells; focus outlines.

2.2 Viper (Snake)

- **UI Layout:** 400×400px HTML Canvas; grid lines optional; dynamic scoreboard overlay in top-right.
- **Core Logic:** Snake as array of coordinate segments; move vector per frame; collision detection on walls/self.
- **Features:** Multiple speed levels (Easy/Medium/Hard), food spawning, optional wrap-around mode.
- **Controls:** Arrow/WASD; swipe gestures on mobile.
- **Scoring:** +10 per food; speed multiplier for combos; display current & high score live.
- **Theme:** Snake = #10B981 (green), food = #EF4444 (red); grid background #F3F4F6.
- **Accessibility:** High-contrast mode toggle.

2.3 Brick Breaker

- **UI Layout:** Canvas 480×320px; paddle at bottom; brick rows at top.
- **Core Logic:** Ball physics with velocity vector; collision math for bricks/paddle/walls.
- **Features:** Multiple brick types (standard, durable), power-ups (expand paddle, multi-ball), level progression.
- **Controls:** Mouse movement / drag; arrow keys.
- **Scoring:** +5 per brick; extra +50 on level clear.
- **Theme:** Bricks palette (#6366F1, #F59E0B, #10B981); paddle #3B82F6; ball #F87171.
- **Accessibility:** Adjustable ball speed and paddle size.

2.4 Pac-Maze

- **UI Layout:** 500×500px canvas; maze walls thick #374151; overlay score & lives above.
- **Core Logic:** Tile-based movement; ghost AI with simple chase modes; pellet collection.
- **Features:** Power pellets, multiple levels, ghost respawn timers.
- **Controls:** Arrow keys / swipe.
- **Scoring:** Pellets=10, power pellets=50, ghost=200+.
- **Theme:** Pac-Man #FBBF24, ghosts mix (#EF4444, #3B82F6, #10B981, #A855F7).
- **Accessibility:** Pause overlay with alt-text descriptions.

2.5 Snakes & Ladders

- **UI Layout:** 10×10 grid board image; draggable token components.

- **Core Logic:** Virtual dice (1–6), token movement, snake/ladder links map.
- **Features:** Up to 4 players; choose token color; animated moves.
- **Controls:** Click “Roll Dice”; token moves automatically.
- **Scoring:** Track turns taken; fastest win stats.
- **Theme:** Board accents (#F97316, #3B82F6); tokens colored.
- **Accessibility:** Dice roll announced via screen reader.

2.6 Ludo

- **UI Layout:** Standard Ludo cross board; tokens group in nests; center home cluster.
- **Core Logic:** Dice roll, token exit on 6, safe squares, knockout logic.
- **Features:** 2–4 players local; token selection highlight; end-game summary.
- **Controls:** Roll button + click token to move.
- **Scoring:** Time to finish; number of knockouts.
- **Theme:** Quadrant colors (#EF4444, #3B82F6, #10B981, #F59E0B).
- **Accessibility:** Color-blind mode with patterns on tokens.

2.7 Pong

- **UI Layout:** Canvas 640×480; two paddles at sides; ball center; score large at top.
- **Core Logic:** Velocity vector, bounce physics; AI paddle logic at medium difficulty.
- **Features:** Single vs AI / two-player, adjustable ball speed.
- **Controls:** W/S & Arrow up/down; touch arcs on mobile sides.
- **Scoring:** First to 10 wins; match history saved.
- **Theme:** Paddles #3B82F6; ball #FBBF24; background #111827.
- **Accessibility:** Sound cue on hit/out-of-bounds.

2.8 Connect Four

- **UI Layout:** 7×6 grid; column hover highlight; drop animation.
- **Core Logic:** 2D array, check for 4-alignment in all directions.
- **Features:** Undo last move; AI opponent (minimax depth 4).
- **Controls:** Click/tap column; keyboard navigation.
- **Scoring:** Win/loss record.
- **Theme:** Disc colors #EF4444 vs #3B82F6; board #F3F4F6.
- **Accessibility:** Voice cues for drop.

2.9 Minesweeper

- **UI Layout:** Grid (variable size: beginner/medium/expert); cell reveal animation.
- **Core Logic:** Flood-fill for zero cells; number adjacency counts; flagging.
- **Features:** Timer, flag counter, difficulty presets.
- **Controls:** Left-click reveal; right-click flag; long-press on mobile.

- **Scoring:** Time to clear; best times per difficulty.
- **Theme:** Cell default #D1D5DB; numbers colored (#3B82F6, #EF4444, etc.).
- **Accessibility:** Keyboard focus and enter to reveal.

2.10 Memory Match

- **UI Layout:** Grid of face-down cards (4×4 default); flip animation.
 - **Core Logic:** Track two-card selections; match check; hide on match or flip back.
 - **Features:** Multiple themes (animals, symbols), adjustable grid size.
 - **Controls:** Click/tap; keyboard navigation + Enter.
 - **Scoring:** Moves count; timer; star rating based on efficiency.
 - **Theme:** Card backs #3B82F6; matched highlight #A78BFA.
 - **Accessibility:** Alt-text on cards; high-contrast option.
-

3. Detailed Feature Requirements Detailed Feature Requirements

3.1. User Authentication & Profiles (Local-First)

- **Profile Creation:** Unique displayName + avatarId (1 of 20 SVGs). No passwords.
- **Login Selection:** Choose existing profile from local JSON list.
- **Guest Access:** Play without saving scores.

3.2. Main User Dashboard

- **Welcome Banner:** “Welcome back, [DisplayName]!”
- **Overall Stats:** Total Games Played, Win/Loss %.
- **Per-Game Stats:** High scores + play counts grid.
- **Recent Activity:** Last 5 sessions feed.

3.3. Main UI & Game Lobby

- **Layout:** Two-column—left sidebar for nav, main content area.
- **Sidebar:** Icons & labels for Dashboard, each game, Leaderboards.
- **Content:** On game select, show Start screen with description + Play button + controls hint.

3.4. Individual Game Experience

- **Rendering:** HTML Canvas via `requestAnimationFrame`.
- **Sound:** Tone.js for music & effects.
- **Controls:** Keyboard (desktop) + touch UI (mobile).
- **In-Game UI:** Live score, personal best, pause button.
- **Game Over:** Final score, personal best, Play Again, Back to Lobby, new-high-score notification.

3.5. Leaderboards

- **Per-Game Leaderboard:** Top 20 all-time scores.
 - **Display:** Rank, avatar, `displayName`, score, date.
 - **Highlight:** Current user's rank.
-

4. UI/UX Design Guidelines

- **Theme:** White (#FFFFFF) / light gray (#F4F4F5) with bright accents (#3B82F6).
 - **Typography:** Inter or similar sans-serif.
 - **Components:** Rounded corners (8px), subtle shadows, solid buttons, hover states.
 - **Animations:** Smooth transitions for interactions.
 - **Responsiveness:** Fluid layouts for all screen sizes.
-

5. Technical Requirements

5.1. Frontend

- **Tech:** HTML, CSS (Tailwind via CDN), vanilla JS (ES6+).
- **Routing:** Hash-based routing (`#dashboard`, `#game-viper`, `#leaderboards`).

- **Components:** Modular JS + CSS classes; use GameCard, ScoreTable, AvatarPicker naming conventions.

5.2. Backend & Data (Local)

- **Server:** json-server on db.json (port 3001).

- **Data Model:**

```
{
  "users": [ { "id": 1, "displayName": "PlayerOne", "avatarId":
"avatar5.svg", "stats": { "totalGamesPlayed":50,"wins":20,"losses":30 }
} ],
  "scores": [ { "id":1, "userId":1, "gameId":"viper", "score":1500,
"timestamp":"2025-06-26T10:00:00Z" } ]
}
```

- **Backup:** Export/import JSON via UI for persistence beyond browser clear.

5.3. Non-Functional

- **Performance:** <1s load, 60 fps gameplay.
- **Compatibility:** Chrome, Firefox, Safari, Edge (latest).
- **Scalability:** Abstract API calls via api.js to swap to IndexedDB/localForage later.

6. Project Structure

```
/retro-oasis/
├── index.html
├── db.json           # json-server data
├── package.json
├── /src/
│   ├── /js/
│   │   ├── main.js      # routing & init
│   │   ├── api.js       # fetch wrapper
│   │   ├── ui.js        # UI helpers
│   │   └── /games/
│   │       ├── interface.js # standard game API
│   │       ├── tic-tac-toe.js
│   │       └── ...        # one file per game
│   ├── /css/
│   │   └── style.css
│   └── /assets/
│       ├── /avatars/    # avatar1.svg ... avatar20.svg
│       └── /sound/       # optional audio files
```

```
|      | /images/      # logo.svg, icons
|      | README.md      # setup + endpoints
```

7. Milestones

- **M1 (MVP):** Setup repo + json-server; UI shell; Tic-Tac-Toe, Viper, Brick Breaker; profile select; score save.
 - **M2:** Dashboard; Leaderboards; remaining 7 games; avatar picker.
 - **M3:** Tone.js integration; UI animations; mobile polish; testing & bug-fixing.
-

8. Game Module Interface

All game modules must export:

```
export function init(containerElement)  // prepare DOM/canvas
export function startGame()             // begin or resume game
export function pauseGame()             // pause logic
export function resetGame()             // clear state
export function getCurrentScore() -> int // for Game Over screen
```

This ensures consistent lifecycle management across games.

9. API Endpoint Reference

Meth od	Endpoint	Description
GET	/users	List all user profiles
POST	/users	Create new user profile
GET	/scores?gameId=<id>&_sort=score&_order=desc	List scores, sorted desc by score
POST	/scores	Submit new score
GET	/scores?gameId=<id>&_sort=score&_order=desc&_limit=20	Top 20 scores for a game

10. Local Development Setup

1. `git clone <repo-url>`
2. `npm install -g json-server`
3. `json-server --watch db.json --port 3001`
4. Open `index.html` in browser or run `npx live-server`
5. Start hacking!

Now your document is ready for VS Code and GitHub Copilot to scaffold a robust local-first gaming platform!